

Penguins Regression with Auto-Sklearn 2 + Cross-Validation

This notebook demonstrates how to use **Auto-Sklearn 2** (an AutoML library) to predict penguin body mass (`body_mass_g`) from the classic Palmer Penguins dataset.

Rather than a simple train/test split, we use **K-Fold Cross-Validation** across the entire dataset to produce robust, out-of-fold (OOF) performance estimates. A final model is then trained on all available data for deployment.

Workflow overview:

1. Load and inspect the dataset
2. Clean missing values
3. Encode categorical features
4. Run K-Fold CV with an AutoML regressor per fold
5. Fit a final model on the full dataset

Regression with Auto-Sklearn 2
Clean data
Encode features
K-Fold or Stratified K-Fold
For each fold: Train AutoSklearnRegressor
Predict on validation
Collect OOF (out-of-fold) results
Compute metrics (R^2 , RMSE, etc.)
After CV, train final model on all data

Configuration & Imports

We import all required libraries upfront and define a small set of hyperparameters that control the cross-validation and AutoML search:

Parameter	Value	Purpose
<code>k</code>	7	Number of CV folds
<code>time_limit</code>	120 s	AutoML search budget per fold
<code>random_state</code>	42	Reproducibility seed for KFold shuffling

Note: `auto_sklearn2` wraps many sklearn-compatible models and automatically selects the best pipeline via Bayesian optimisation within the given time budget.

```
In [1]: # Penguins regression with auto_sklearn2 + cross-validation (no train/test split)
# Predict target: body_mass_g
from auto_sklearn2 import AutoSklearnRegressor
import seaborn as sns
import pandas as pd
import numpy as np
from sklearn.model_selection import KFold # no train_test_split
```

```

from sklearn.metrics import r2_score, mean_squared_error

# -----
# Config
# -----
k = 7          # number of CV folds
time_limit = 120 # seconds per AutoML fit (per fold)
#random_state = 42 # reproducibility for KFold shuffling

```

1. Load Dataset

We load the **Palmer Penguins** dataset directly from Seaborn's built-in datasets. It contains physical measurements for three penguin species (Adelie, Chinstrap, Gentoo) collected from three islands in Antarctica.

Our regression **target** is `body_mass_g` — the penguin's body mass in grams. All other columns will serve as features.

```

In [3]: # -----
# 1) Load dataset
# -----
penguins = pd.read_csv('./data/penguins_synthetic_5000.csv')

# Target
target_col = "body_mass_g"
df = penguins.copy()

```

2. Basic Cleaning — Drop Missing Values

The dataset contains a small number of rows with missing values across various columns. For simplicity, we **drop all rows with any NaN** values.

This is a conservative approach that ensures every sample fed to the model is complete. In production you might prefer imputation strategies, but for this dataset the data loss is minimal.

```

In [4]: # -----
# 2) Basic cleaning: drop rows with missing values
# -----
df = df.dropna()

```

3. One-Hot Encoder - Categorical Features

Machine learning models require numerical inputs. The dataset has categorical columns (e.g. `species`, `island`, `sex`) that must be converted to numbers.

We use **one-hot encoding** (`pd.get_dummies`) with `drop_first=True` to avoid the **dummy variable trap** (perfect multicollinearity), where one category can always be inferred from the others.

After encoding, we separate the feature matrix `X` and the target vector `y`. Indices are reset so they align correctly with the KFold index arrays generated later.

```
In [5]: # -----
# 3) One-hot encode categoricals
# -----
feature_cols = df.drop(columns=[target_col])
numeric_features = feature_cols.select_dtypes(include=np.number).columns
categorical_features = feature_cols.select_dtypes(exclude=np.number).columns

df_enc = pd.get_dummies(df, columns=categorical_features, drop_first=True)

# Split X / y (no holdout; we will do CV over the whole dataset)
X = df_enc.drop(columns=[target_col])
y = df_enc[target_col].astype(float).reset_index(drop=True)
X = X.reset_index(drop=True)
```

4. K-Fold Cross-Validation with AutoML

Instead of a fixed train/test split, we use **k-Fold Cross-Validation** over the *entire* dataset. This means:

- The data is split into k roughly equal folds.
- In each iteration, $k-1$ folds are used for training and 1 fold is held out for validation.
- Every row is used for validation exactly once → **out-of-fold (OOF) predictions**.

A **fresh** `AutoSklearnRegressor` is trained in each fold, allowing the AutoML search to find the best pipeline independently per fold. We record R^2 and RMSE per fold, then aggregate.

Metrics used:

- **R^2 (coefficient of determination)**: Proportion of variance explained; 1.0 is perfect.
- **RMSE (root mean squared error)**: Average prediction error in the same units as the target (grams).

The OOF predictions are stitched back together to give a single, unbiased estimate of model performance across the full dataset.

```
In [6]: # -----
# 4) K-fold cross-validation over the entire dataset
# We train a fresh AutoSklearnRegressor in each fold and compute OOF metrics.
# -----
kf = KFold(n_splits=k, shuffle=True) # , random_state=random_state
```

```

cv_r2 = []
cv_rmse = []

# Out-of-fold predictions container (same order as y)
oof_pred = np.full(shape=len(y), fill_value=np.nan, dtype=float)

for fold, (tr_idx, va_idx) in enumerate(kf.split(X), start=1):
    X_tr, X_va = X.iloc[tr_idx], X.iloc[va_idx]
    y_tr, y_va = y.iloc[tr_idx], y.iloc[va_idx]

    # A smaller time budget per fold keeps total runtime bounded
    automl_cv = AutoSklearnRegressor(time_limit=time_limit)
    automl_cv.fit(X_tr, y_tr)

    y_va_pred = automl_cv.predict(X_va)
    oof_pred[va_idx] = y_va_pred

    r2 = r2_score(y_va, y_va_pred)
    rmse = np.sqrt(mean_squared_error(y_va, y_va_pred))
    cv_r2.append(r2)
    cv_rmse.append(rmse)
    print(f"[Fold {fold}] R²={r2:.4f} | RMSE={rmse:.2f}")

# Summary across folds
print(f"\n=== Cross-Validation Summary ({k}-fold) ===")
print(f"R² Mean: {np.mean(cv_r2):.4f} | R² Std: {np.std(cv_r2):.4f}")
print(f"RMSE Mean: {np.mean(cv_rmse):.2f} | RMSE Std: {np.std(cv_rmse):.2f}")

# Out-of-fold performance: single estimate over all rows using OOF predictions
oof_r2 = r2_score(y, oof_pred)
oof_rmse = np.sqrt(mean_squared_error(y, oof_pred))
print(f"\n=== Out-of-Fold (OOF) Performance on Full Dataset ===")
print(f"OOF R²: {oof_r2:.4f} | OOF RMSE: {oof_rmse:.2f}")

```

```

[Fold 1] R²=0.9987 | RMSE=29.51
[Fold 2] R²=0.9984 | RMSE=31.25
[Fold 3] R²=0.9982 | RMSE=34.26
[Fold 4] R²=0.9987 | RMSE=29.93
[Fold 5] R²=0.9979 | RMSE=36.97
[Fold 6] R²=0.9985 | RMSE=31.01
[Fold 7] R²=0.9976 | RMSE=38.95

```

```

=== Cross-Validation Summary (7-fold) ===
R² Mean: 0.9983 | R² Std: 0.0004
RMSE Mean: 33.13 | RMSE Std: 3.41

```

```

=== Out-of-Fold (OOF) Performance on Full Dataset ===
OOF R²: 0.9983 | OOF RMSE: 33.30

```

5. Final Model — Trained on the Full Dataset

Once we have a reliable CV performance estimate, we train a **final model on 100% of the available data**. This is standard practice:

- More training data generally means a better model.

- We already have an unbiased performance estimate from the OOF step, so no separate test set is needed.
- This final model is the one you would save and use for predictions on new, unseen penguins.

⚠ **Do not evaluate this final model on the training data** — the OOF R^2 /RMSE from Step 4 are your true generalisation estimates.

```
In [7]: # -----
# 5) Fit final model on the full dataset (for deployment)
# No holdout; you rely on CV/OOF for generalization estimates.
# -----
auto_sklern_full = AutoSklernRegressor(time_limit=time_limit)
auto_sklern_full.fit(X, y)

# Try to report best params if available
best_params = getattr(auto_sklern_full, "best_params", None)
if best_params is None:
    best_params = getattr(auto_sklern_full, "best_params_", "N/A")

print("\n=== Final Model Trained on Full Data ===")
print(f"Best params: {best_params}")
```

```
=== Final Model Trained on Full Data ===
Best params: {'preprocessor': 'robust_scaler', 'regressor': 'knn'}
```

6. (Optional) Model Leaderboard

If the installed version of `auto_sklern2` exposes a `get_models_performance()` method, we can inspect the **leaderboard** — a ranked table of all models that were tried during the AutoML search, along with their scores.

This is useful for understanding which model families (e.g. gradient boosting, random forests, linear models) the AutoML search favoured, and by how much they differ in performance.

```
In [8]: # Optional: show leaderboard if provided by auto_sklern2
if hasattr(auto_sklern_full, "get_models_performance"):
    print("\nModel leaderboard (auto_sklern2):")
    perf = auto_sklern_full.get_models_performance()
    df_lb = pd.DataFrame(list(perf.items()), columns=["model", "score"])
    df_lb.sort_values("score", ascending=False, inplace=True)
    print(df_lb.head(10))
else:
    print("\n(Leaderboard API not available in this version.)")
```

```
Model leaderboard (auto_sklearn2):
```

	model	score
51	robust_scaler_knn	0.998044
7	standard_scaler_knn	0.997973
29	minmax_scaler_knn	0.997951
11	standard_scaler_extra_trees	0.996264
33	minmax_scaler_extra_trees	0.996113
55	robust_scaler_extra_trees	0.995910
0	standard_scaler_random_forest	0.993713
22	minmax_scaler_random_forest	0.993485
44	robust_scaler_random_forest	0.993402
56	robust_scaler_bagging	0.993142

In []: